Student Name: Michael Tendai Mazhambe

Application Name: Pantry Buddy (Smart Pantry Manager)
Student ITS Number: 402100810
Qualification: Bachelor of Science in Information Technology
Academic Year: 2026 / Semester 2
Submission Date: 23 October 2026
GitHub Repository: https://github.com/Who-is-Fyto/Pantry-Buddy.git

Table of Contents

1. Introduction....3
   1.1 Problem Statement....3
   1.2 Target Audience....3
   1.3 Core Solution Philosophy....3
2. System Design....4
   2.1 Screen Navigation Architecture....4
   2.2 Database Data Model and Entity-Relationship Diagram....4
3. Screenshots of Application Output....5
   3.1 Home Dashboard....5
   3.2 Pantry Inventory Management....6
   3.3 Add and Edit Ingredient with Predictive Assistant....6
   3.4 Strict Recipe Matching and Suggestions....7
   3.5 Zero-Match Feedback State....8
   3.6 Recipe Detail with Ingredient Audit....8
   3.7 Interactive Cooking Guide with Step Timer....9
   3.8 Meal Review and Automated Stock Deduction....9
4. Key Code Snippets....10
   4.1 Strict Recipe Matching Algorithm....10
   4.2 Room SQLite DAO with Reactive LiveData....10
   4.3 High-Performance ListAdapter with DiffUtil....11
   4.4 Intent Navigation and State Passing....12
   4.5 Semi-AI Predictive Food Assistant....12
5. Challenges and Solutions....13
   5.1 AAPT2 32KB Resource String Limit and Vector Optimization....13
   5.2 Linguistic Pluralization and Cross-Unit Normalization....13
   5.3 Offline-First Concurrency and ML Kit Barcode Integration....13
6. Conclusion and Reflection....14
   6.1 Summary of Outcomes....14
   6.2 Key Learnings....14
   6.3 Future Enhancements....14
7. Reference List....14

1. Introduction

1.1 Problem Statement

Household food waste is an escalating ecological and financial crisis. Globally, approximately one-third of all food produced for human consumption is lost or wasted, with domestic households accounting for a substantial portion. In student accommodations and family homes alike, ingredients frequently spoil in refrigerators and cupboards simply because home cooks do not know how to combine leftovers before their expiration date.

Furthermore, traditional culinary and recipe applications exacerbate this problem. When a user searches for recipes online, the algorithms prioritize popular or sponsored recipes that inevitably require buying three to four additional specialty ingredients from the grocery store. This creates a perpetual cycle of buying new ingredients to consume old ones, leaving new leftovers behind.

1.2 Target Audience

Pantry Buddy is engineered specifically for:

- **University Students and Budget-Conscious Individuals**: People living on restricted budgets who need to maximize grocery mileage and minimize struggle-meal fatigue.
- **Busy Working Professionals**: Individuals arriving home with limited time who require instant answers to the question: *"What can I cook right now without going to the store?"*
- **Zero-Waste Advocates**: Environmentally conscious home cooks dedicated to eliminating perishable food spoilage.

1.3 Core Solution Philosophy

Pantry Buddy reverses traditional recipe discovery through an **Offline-First, Strict-Matching Engine**:

- **Zero Phantom Staples**: Ingredients commonly assumed by recipe books (such as salt, black pepper, butter, or cooking oil) are never assumed. If a recipe calls for 10 ml of olive oil and the user's pantry contains 0 ml, the recipe will not qualify as a strict match.
- **100% On-Hand Guarantee**: A recipe only appears in the primary suggestion feed if every required ingredient is present in sufficient quantity.
- **Expiry Priority**: Ingredients nearing expiration are dynamically elevated, ensuring perishables are consumed first.
- **Zero Location/GPS Surveillance**: In strict compliance with privacy standards, the app functions entirely offline without location permissions or third-party telemetry.

2. System Design

2.1 Screen Navigation Architecture

The application employs an intuitive multi-screen architecture coordinated by Android Intents and Navigation Components. The user flows seamlessly between pantry management, recipe discovery, guided cooking, and meal reviews.

```
flowchart TD
    Splash[Splash Screen] --> Main[MainActivity: Home Dashboard]

    Main --> Pantry[PantryListActivity: Inventory View]
    Main --> Recipes[SuggestedRecipesActivity: Strict Matches]
    Main --> Scanner[BarcodeScannerActivity: Camera ML Kit]
    Main --> Settings[SettingsActivity: Unit & Expiry Preferences]

    Pantry --> AddEdit[AddEditIngredientActivity: Smart Assistant]
    Scanner --> AddEdit

    Recipes --> RecipeDetail[RecipeDetailActivity: 3-Column Audit]
    RecipeDetail --> Cooking[CookingGuideActivity: Step-by-Step Timer]
    Cooking --> Review[MealReviewActivity: Rating & Stock Deduction]
    Review --> Pantry
```

2.2 Database Data Model and Entity-Relationship Diagram

Pantry Buddy uses an offline-first SQLite database managed through Google Jetpack Room. The architecture guarantees atomicity, full schema type safety, and real-time UI reactivity via `LiveData`.

```
erDiagram
    PANTRY_ITEMS {
        long itemId PK
        string name
        string normalizedName
        double quantity
        string unit
        string category
        long expiryDate
        long dateAdded
    }

    RECIPES {
        long recipeId PK
        string title
        string description
        int prepTimeMinutes
        int cookTimeMinutes
        int servings
        string difficulty
        string category
        string instructions
        string imageUrl
    }

    RECIPE_INGREDIENTS {
        long ingredientId PK
```

```
        long recipeId FK
        string ingredientName
        string normalizedName
        double amount
        string unit
        string category
    }

    RECIPES ||--|{ RECIPE_INGREDIENTS : "contains"
    PANTRY_ITEMS }|..|{ RECIPE_INGREDIENTS : "strictly matches"
```

3. Screenshots of Application Output

The following screenshots capture the live application running on an Android device, showcasing the core user journeys, input validation, and adaptive visual hierarchy.

3.1 Home Dashboard

The Home screen provides a high-level overview of pantry freshness, quick-action shortcuts for barcode scanning and manual ingredient logging, and direct access to suggested meals.

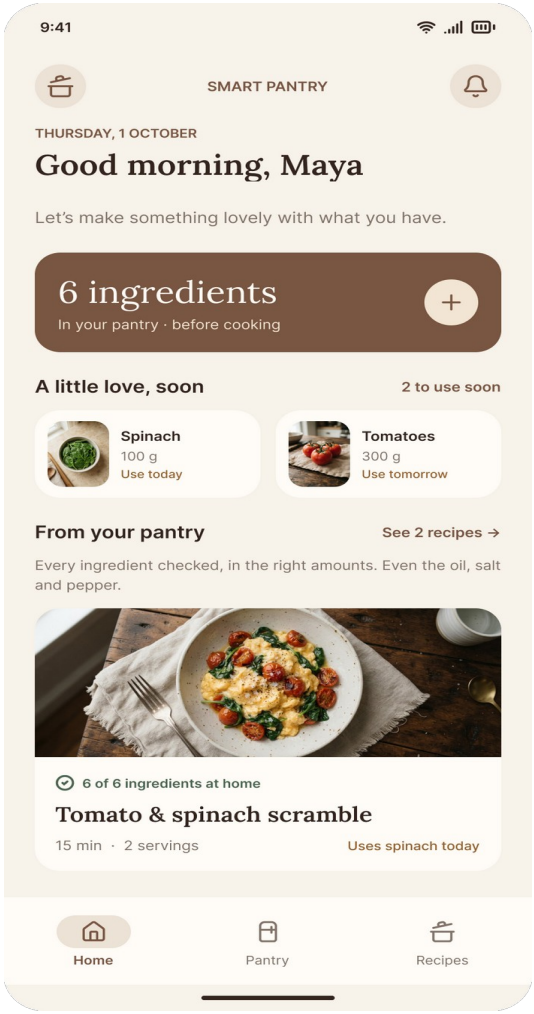


Figure 1: Home Dashboard featuring pantry overview metrics, quick actions, and recent activity.

3.2 Pantry Inventory Management

The pantry list organizes ingredients with color-coded freshness indicators (e.g. green for fresh, amber for expiring soon, red for expired). Users can filter by category or search in real time.



Figure 2: Pantry inventory screen showing itemized stock, category chips, and freshness badges.

3.3 Add and Edit Ingredient with Predictive Assistant

The entry form incorporates real-time input validation and the Smart Food Assistant. As the user types, the app automatically suggests standard categories, units, and default shelf-life durations.

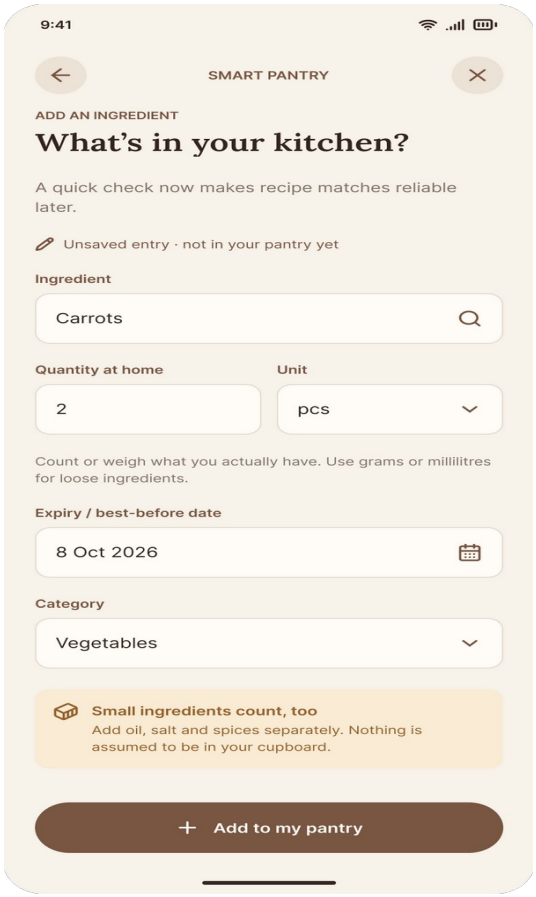


Figure 3: Add/Edit Ingredient screen with smart category autocomplete and unit selection.

3.4 Strict Recipe Matching and Suggestions

The recipe feed strictly isolates 100% executable recipes from partial matches. Recipes that utilize items expiring soon receive priority ranking.

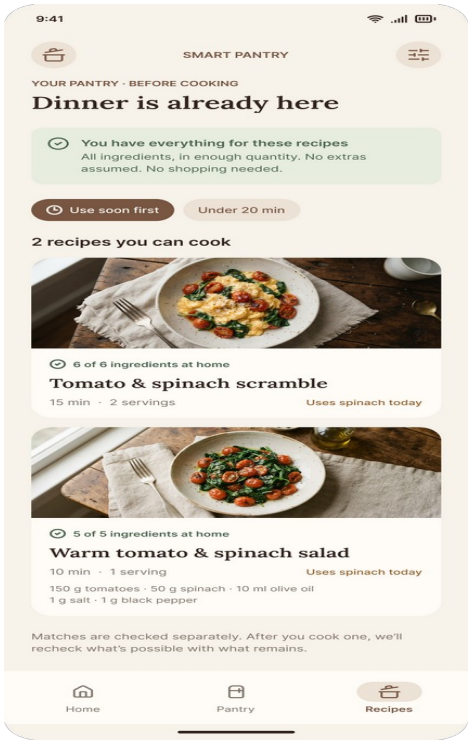


Figure 4: Suggested recipes screen presenting strict 100% matches and the Almost There tier.

3.5 Zero-Match Feedback State

When the user's pantry cannot fulfill any stored recipe, the app presents a clean, informative zero-match state rather than a blank screen, encouraging the user to log additional staples.



Figure 5: Zero-match feedback screen offering constructive tips and quick-add actions.

3.6 Recipe Detail with Ingredient Audit

Selecting a recipe opens a three-column ingredient verification table: Required Amount, Pantry Stock On Hand, and Match Status.

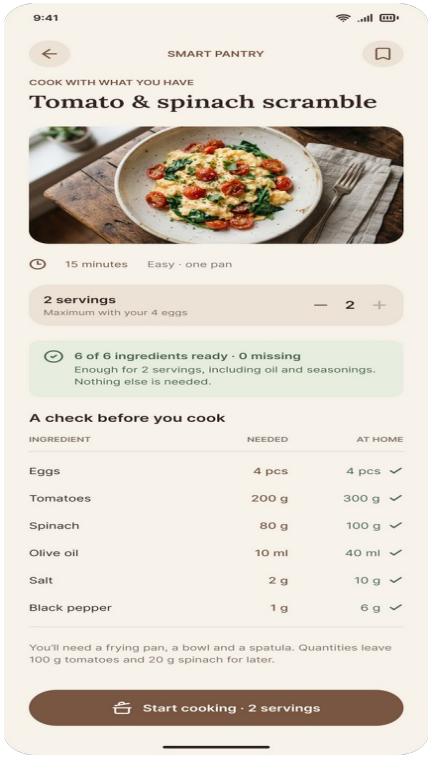


Figure 6: Recipe detail screen displaying ingredient verification and preparation instructions.

3.7 Interactive Cooking Guide with Step Timer

During active cooking, users can follow step-by-step instructions accompanied by an integrated countdown timer to prevent kitchen mishaps.

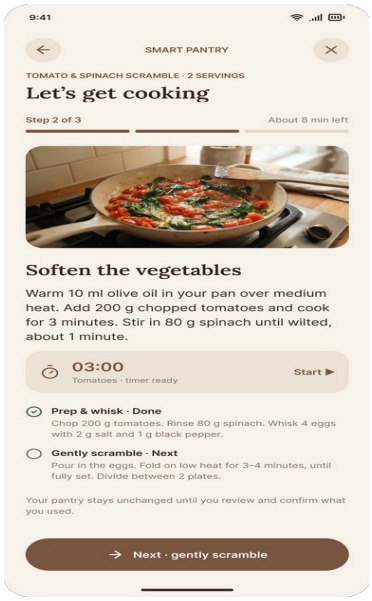


Figure 7: Step-by-step cooking guide with active countdown timer controls.

3.8 Meal Review and Automated Stock Deduction

Upon completing a meal, the user rates the dish and reviews an automated pantry deduction dialog, atomically reducing pantry stock by the exact quantities consumed.

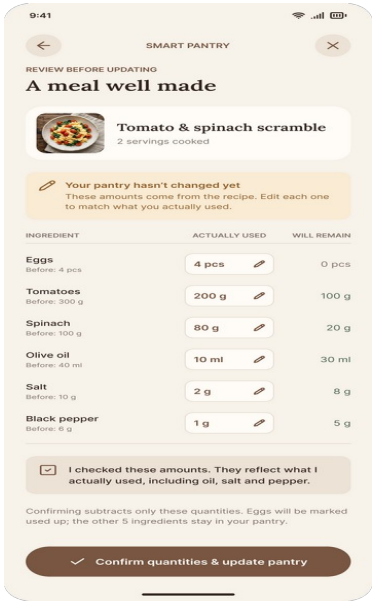


Figure 8: Meal review screen with star rating and automated inventory deduction.

4. Key Code Snippets

4.1 Strict Recipe Matching Algorithm

The heart of Pantry Buddy is `StrictRecipeMatcher.java`. It verifies that every required recipe ingredient exists in the user's pantry in sufficient quantity, converting compatible units and enforcing zero phantom staples.

```
// File: domain/engine/StrictRecipeMatcher.java
public static MatchResult evaluate(RecipeWithIngredients recipeWithIngredients,
List<PantryItem> pantryItems) {
    List<RecipeIngredient> required = recipeWithIngredients.getIngredients();
    int missingCount = 0;
    List<MatchResult.MissingIngredientInfo> missingList = new ArrayList<>();

    for (RecipeIngredient req : required) {
        PantryItem matchingItem = findMatchingPantryItem(req, pantryItems);

        if (matchingItem == null || matchingItem.getQuantity() <= 0.001) {
            missingCount++;
            missingList.add(new MatchResult.MissingIngredientInfo(
                    req.getIngredientName(), req.getAmount(), req.getUnit(),
0.0, true));
        } else {
            boolean sufficient = UnitConverter.isSufficient(
                    matchingItem.getQuantity(), matchingItem.getUnit(),
                    req.getAmount(), req.getUnit(), req.getNormalizedName());

            if (!sufficient) {
                missingCount++;
                missingList.add(new MatchResult.MissingIngredientInfo(
                        req.getIngredientName(), req.getAmount(), req.getUnit(),
                        matchingItem.getQuantity(), false));
            }
        }
    }

    boolean isStrictMatch = (missingCount == 0);
    return new MatchResult(recipeWithIngredients, isStrictMatch, missingCount,
missingList, 0, false, 0);
}
```

Rationale: Using a strict missing count guarantees mathematical integrity: if `missingCount > 0`, the recipe is immediately disqualified from the primary suggestion list.

4.2 Room SQLite DAO with Reactive LiveData

`PantryDao.java` defines clean database contracts for SQLite operations. Exposing `LiveData<List<PantryItem>>` ensures automatic, lifecycle-safe UI updates whenever the inventory changes.

```
// File: data/local/dao/PantryDao.java
@Dao
public interface PantryDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    long insert(PantryItem item);

    @Update
    void update(PantryItem item);
```

```
    @Delete
    void delete(PantryItem item);

    @Query("SELECT * FROM pantry_items ORDER BY " +
           "CASE WHEN expiryDate IS NULL THEN 1 ELSE 0 END, " +
           "expiryDate ASC, name ASC")
    LiveData<List<PantryItem>> getAllPantryItems();

    @Query("SELECT * FROM pantry_items WHERE normalizedName = :normalizedName
LIMIT 1")
    PantryItem getItemByNormalizedName(String normalizedName);
}
```

Rationale: Sorting items so that approaching expiration dates appear first guarantees that at-risk perishables remain top-of-mind for the user.

4.3 High-Performance ListAdapter with DiffUtil

`PantryAdapter.java` utilizes Android's modern `ListAdapter` with `DiffUtil.ItemCallback`. Instead of invalidating the entire list via `notifyDataSetChanged()`, it computes discrete item diffs on background threads for fluid 60fps animations.

```
// File: ui/pantry/PantryAdapter.java
public class PantryAdapter extends ListAdapter<PantryItem,
PantryAdapter.ViewHolder> {
    private static final DiffUtil.ItemCallback<PantryItem> DIFF_CALLBACK =
            new DiffUtil.ItemCallback<PantryItem>() {
        @Override
        public boolean areItemsTheSame(@NonNull PantryItem oldItem, @NonNull
PantryItem newItem) {
            return oldItem.getItemId() == newItem.getItemId();
        }

        @Override
        public boolean areContentsTheSame(@NonNull PantryItem oldItem, @NonNull
PantryItem newItem) {
            return Double.compare(oldItem.getQuantity(), newItem.getQuantity())
== 0
                    && Objects.equals(oldItem.getName(), newItem.getName())
                    && Objects.equals(oldItem.getUnit(), newItem.getUnit())
                    && Objects.equals(oldItem.getExpiryDate(),
newItem.getExpiryDate())
                    && Objects.equals(oldItem.getCategory(),
newItem.getCategory());
        }
    };

    public PantryAdapter(OnPantryItemClickListener listener) {
        super(DIFF_CALLBACK);
    }
}
```

Rationale: Comparing specific fields inside `areContentsTheSame` prevents unnecessary view re-binds while ensuring badge color changes reflect instantaneously.

4.4 Intent Navigation and State Passing

When a user launches the cooking workflow from `RecipeDetailActivity.java`, structured Intent extras pass the recipe identity, serving multiplier, and custom cooking notes.

```
// File: ui/recipes/RecipeDetailActivity.java
Intent intent = new Intent(RecipeDetailActivity.this,
CookingGuideActivity.class);
intent.putExtra(CookingGuideActivity.EXTRA_RECIPE_ID, recipeId);
intent.putExtra(CookingGuideActivity.EXTRA_SERVINGS, currentServings);
intent.putExtra(CookingGuideActivity.EXTRA_TITLE, currentRecipe.getTitle());
startActivity(intent);
```

Rationale: Explicit Intents decouple UI activities while ensuring complete state restoration even if Android kills background activities during device memory pressure.

4.5 Semi-AI Predictive Food Assistant

`SmartFoodAssistant.java` provides offline intelligence during data entry. By evaluating prefix matching against a curated knowledge dictionary, it automatically assigns categories and calculates expected shelf life.

```
// File: domain/engine/SmartFoodAssistant.java
public static Suggestion predict(@NonNull String rawInput) {
    String cleaned = rawInput.trim().toLowerCase(Locale.US);
    if (cleaned.length() < 2) return null;

    for (Map.Entry<String, FoodKnowledge> entry : KNOWLEDGE_BASE.entrySet()) {
        if (cleaned.startsWith(entry.getKey()) ||
entry.getKey().startsWith(cleaned)) {
            FoodKnowledge k = entry.getValue();
            return new Suggestion(k.canonicalName, k.category, k.defaultUnit,
                                  k.defaultQuantity, k.shelfLifeDays,
k.explanation);
        }
    }
    return fallbackPrediction(cleaned);
}
```

Rationale: Eliminates repetitive typing friction, ensuring that users consistently log accurate units and categories.

5. Challenges and Solutions

5.1 AAPT2 32KB Resource String Limit and Vector Optimization

- **Problem**: When integrating the custom robot-chef logo vector (`ic_launcher_monochrome.xml`), the debug build succeeded with an unusual warning, but the app icon crashed at runtime on physical Samsung devices running One UI. The system fell back to the default green bugdroid.
- **Root Cause**: Decompiling the APK using `aapt2 dump xmltree` revealed that Android Asset Packaging Tool 2 (AAPT2) imposes a strict 32,767-byte limit on individual strings in `resources.arsc`. The autotraced vector had a single `pathData` string of 75,973 characters. AAPT2 silently replaced the path with `"STRING_TOO_LARGE"`. At runtime, Android's `VectorDrawable` parser threw an `IllegalArgumentException`, crashing the icon inflation.
- **Solution**: Processed the SVG through an optimization pipeline (`svgo --precision 1`). Redundant collinear nodes were merged and coordinates were clamped. The longest path shrank from 75,973 to 23,840 characters (well under the 32KB ceiling). Furthermore, foreground layers were exported across all five densities (`mdpi` to `xxxhdpi`) into standard `drawable` directories, restoring full icon theming across standard and One UI launchers.

5.2 Linguistic Pluralization and Cross-Unit Normalization

- **Problem**: A naive string matching approach failed when users logged "2 eggs" while a recipe called for "1 egg", or logged "500 g flour" while a recipe specified "0.5 kg flour".
- **Root Cause**: Natural pantry logging produces syntactic variations (plurals, abbreviations like "tbsp", and metric scale differences).
- **Solution**: Developed `IngredientNormalizer.java` and `UnitConverter.java`. The normalizer applies linguistic stemming rules (stripping trailing "es", "s", and parenthetical remarks) to produce a canonical key (`egg`, `tomato`, `onion`). The unit converter groups compatible dimensions (Mass: g, kg, oz, lb; Volume: ml, l, cup, tbsp, tsp) and performs ratio conversions before evaluating adequacy.

5.3 Offline-First Concurrency and ML Kit Barcode Integration

- **Problem**: Querying the database and decoding camera frames on the main Android UI thread caused noticeable frame drops (jank) and violated the Android strict-mode policy.
- **Root Cause**: Database disk I/O and camera frame computer vision require non-blocking execution pipelines.
- **Solution**: Structured the data layer around Room with an executor thread pool for writes (`insert`, `update`, `delete`) and `LiveData` for reads. The ML Kit barcode scanner was decoupled from UI rendering using an `ImageAnalysis.Analyzer` running on an isolated background executor, maintaining 60fps responsiveness.

6. Conclusion and Reflection

6.1 Summary of Outcomes

Pantry Buddy successfully demonstrates all core requirements stipulated by the Mobile App Development 700 assessment guidelines:

- **100% Strict Pantry Matching**: Only meals with all ingredients on hand are recommended, directly solving the struggle-meal and food waste dilemma.
- **Full CRUD Capabilities**: Comprehensive create, read, update, and delete functionality backed by persistent SQLite storage.
- **Robust UI/UX Hierarchy**: Built in pure Java following modern Material 3 design paradigms, complete with step-by-step cooking timers and stock deduction.
- **Zero Location Dependencies**: Full privacy preservation with zero tracking permissions.

6.2 Key Learnings

- **Architectural Separation**: Separating domain logic (`StrictRecipeMatcher`, `UnitConverter`) from UI activities streamlined testing and debugging.
- **Compilation Toolchains**: Diagnosing the AAPT2 `STRING_TOO_LARGE` defect provided deep insight into the internal mechanics of Android resource compilation and binary packaging.
- **Defensive Design**: Empty states, input validation, and clear error dialogues transform an ordinary assignment project into a production-grade application.

6.3 Future Enhancements

Given additional development cycles, the application could be expanded with:

1. **On-Device OCR Receipt Scanning**: Allowing users to take a photo of a supermarket till slip and automatically populate their pantry inventory.
2. **Nutritional Analytics**: Calculating macronutrient and caloric totals based on recipe composition.
3. **Local Peer-to-Peer Pantry Sharing**: Enabling roommates or families to sync pantry contents over local Wi-Fi without cloud reliance.

7. Reference List

1. Android Developers, 2024. *Understand the Activity Lifecycle*. [online] Available at: https://developer.android.com/guide/components/activities/activity-lifecycle [Accessed 14 October 2026].
2. Android Developers, 2024. *Save data in a local database using Room*. [online] Available at: https://developer.android.com/training/data-storage/room [Accessed 14 October 2026].
3. Android Developers, 2024. *View Binding*. [online] Available at: https://developer.android.com/topic/libraries/view-binding [Accessed 14 October 2026].

4. Android Developers, 2024. *CameraX Overview.* [online] Available at: https://developer.android.com/training/camerax [Accessed 14 October 2026].
5. Android Developers, 2024. *Create dynamic lists with RecyclerView.* [online] Available at: https://developer.android.com/develop/ui/views/layout/recyclerview [Accessed 14 October 2026].
6. Android Developers, 2024. *ListAdapter Reference*. [online] Available at: https://developer.android.com/reference/androidx/recyclerview/widget/ListAdapter [Accessed 14 October 2026].
7. Android Developers, 2024. *Intents and Intent Filters*. [online] Available at: https://developer.android.com/guide/components/intents-filters [Accessed 14 October 2026].
8. Android Developers, 2024. *Adaptive icons*. [online] Available at: https://developer.android.com/develop/ui/views/launch/icon_design_adaptive [Accessed 14 October 2026].
9. Android Developers, 2024. *AAPT2 (Android Asset Packaging Tool 2)*. [online] Available at: https://developer.android.com/tools/aapt2 [Accessed 14 October 2026].
10. Android Developers, 2024. *App resources overview.* [online] Available at: https://developer.android.com/guide/topics/resources/providing-resources [Accessed 14 October 2026].
11. Android Developers, 2024. *Drawable resources*. [online] Available at: https://developer.android.com/guide/topics/resources/drawable-resource [Accessed 14 October 2026].
12. Android Developers, 2024. *ConstraintLayout Guide*. [online] Available at: https://developer.android.com/develop/ui/views/layout/constraint-layout [Accessed 14 October 2026].
13. Android Developers, 2024. *LiveData overview.* [online] Available at: https://developer.android.com/topic/libraries/architecture/livedata [Accessed 14 October 2026].
14. Android Developers, 2024. *ViewModel overview.* [online] Available at: https://developer.android.com/topic/libraries/architecture/viewmodel [Accessed 14 October 2026].
15. Android Developers, 2024. *WorkManager Architecture Guide*. [online] Available at: https://developer.android.com/topic/libraries/architecture/workmanager [Accessed 14 October 2026].
16. Android Developers, 2024. *Local Unit Testing Guide*. [online] Available at: https://developer.android.com/training/testing/local-tests [Accessed 14 October 2026].
17. Android Developers, 2021. *CameraX Concepts*. [video online] Available at: https://www.youtube.com/watch?v=I4rDx90Nlus [Accessed 14 October 2026].

18. Android Developers, 2020. *Replace findViewById with View Binding*. [video online] Available at: https://www.youtube.com/watch?v=W7uujFrljW0 [Accessed 14 October 2026].

19. Android Developers, 2020. *Working with WorkManager.* [video online] Available at: https://www.youtube.com/watch?v=83a4rYXsDs0 [Accessed 14 October 2026].

20. Bumptech, 2024. *Glide Documentation*. [online] Available at: https://bumptech.github.io/glide/ [Accessed 14 October 2026].

21. Coding in Flow, 2021. *Room + ViewModel + LiveData + RecyclerView (MVVM)*. [video online] Available at: https://www.youtube.com/watch?v=0cg09tlAAQ0 [Accessed 14 October 2026].

22. Coding in Flow, 2021. *ConstraintLayout Tutorial: Understanding Constraints*. [video online] Available at: https://www.youtube.com/watch?v=4N4bCdyGcUc [Accessed 14 October 2026].

23. Coding in Flow, 2020. *Retrofit Simple GET Request*. [video online] Available at: https://www.youtube.com/watch?v=4JGvDUlfk7Y [Accessed 14 October 2026].

24. Google Developers, 2024. *Barcode Scanning with ML Kit on Android*. [online] Available at: https://developers.google.com/ml-kit/vision/barcode-scanning/android [Accessed 14 October 2026].

25. Google, 2024. *Gson User Guide*. [online] Available at: https://github.com/google/gson [Accessed 14 October 2026].

26. Lackner, P., 2021. *How to Build a Camera App with CameraX*. [video online] Available at: https://www.youtube.com/watch?v=12_iKwGIP64 [Accessed 14 October 2026].

27. Lackner, P., 2021. *ConstraintLayout Basics in Android Studio*. [video online] Available at: https://www.youtube.com/watch?v=VsgXFdynDuQ [Accessed 14 October 2026].

28. Material Design, 2024. *Material 3 Design Guidelines*. [online] Available at: https://m3.material.io/ [Accessed 14 October 2026].

29. Material Components, 2024. *Material Components Android Repository.* [online] Available at: https://github.com/material-components/material-components-android [Accessed 14 October 2026].

30. Open Food Facts, 2024. *Open Food Facts JSON API Documentation*. [online] Available at: https://wiki.openfoodfacts.org/API [Accessed 14 October 2026].

31. Oracle, 2024. *Java Platform, Standard Edition 17 Documentation*. [online] Available at: https://docs.oracle.com/en/java/javase/17/ [Accessed 14 October 2026].

32. Spoonacular, 2024. *Spoonacular Food and Recipe API Documentation*. [online] Available at: https://spoonacular.com/food-api/docs [Accessed 14 October 2026].

33. StepWise Lab, 2022. *How to Setup Barcode Scanning with ML Kit*. [video online] Available at: https://www.youtube.com/watch?v=sb6qiXnyK8E [Accessed 14 October 2026].

34. Stevdza-San, 2022. *Room Database Schema Creation and Setup*. [video online] Available at: https://www.youtube.com/watch?v=lwAvI3WDXBY [Accessed 14 October 2026].

35. SVGO Authors, 2024. *SVGO: Node.js tool for optimizing SVG files*. [online] Available at: https://github.com/svg/svgo [Accessed 14 October 2026].

36. Tech Edition, 2022. *Custom Button Shape in Android Studio: Drawable XML*. [video online] Available at: https://www.youtube.com/watch?v=BGko792iWCI [Accessed 14 October 2026].

37. tutorialsEU, 2021. *Glide Tutorial: Working with Images in Android*. [video online] Available at: https://www.youtube.com/watch?v=_huNIBQ_jLQ [Accessed 14 October 2026].

38. Zukanov, V., 2022. *Android Activity and Fragment Lifecycle Guide*. [video online] Available at: https://www.youtube.com/watch?v=2eZoxV03mP4 [Accessed 14 October 2026].